Practical 2:

Part A:

1. Smart thermostat (Thermostat.java, main in this file)

(a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0.

(b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount.

(c) Constructor(location): chain with this(location, 22).

(d) raise(): if temperature < MAX add 1, else print “Already at maximum (30)”.

(e) lower(): if temperature > MIN subtract 1, else print “Already at minimum (16)”.

(f) Add getTemperature() (no setter) and static getActiveCount().

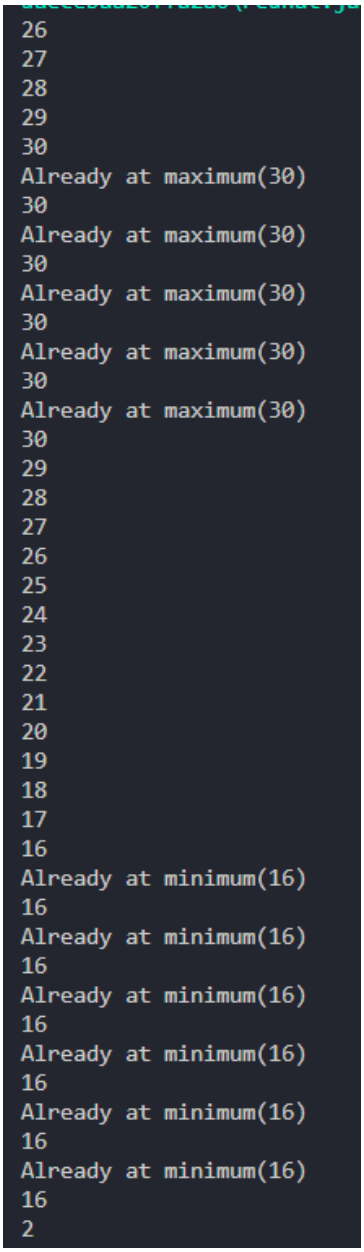
(g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().

Expected output: temperature rises to 30 then “Already at maximum”, falls to 16 then “Already at minimum”, count 2.

Solution:

```java
public class Thermostat{
    private String location;
    private int temperature;
    private static final int MIN=16;
    private static final int MAX=30;
    private static int activeCount=0;
    public Thermostat(String location,int startTemp) {
        this.location = location;
        if(startTemp>=MIN && startTemp<=MAX) {
            this.temperature=startTemp;
        } else{
            this.temperature=22;
        }
        activeCount++;
    }
    public Thermostat(String location) {
        this(location,startTemp: 22);
    }
    public void raise() {
        if (temperature<MAX){
            temperature++;
        } else {
            System.out.println(x: "Already at maximum(30)");
        }
    }
    public void lower() {
        if (temperature>MIN){
            temperature--;
        } else {
            System.out.println(x: "Already at minimum(16)");
        }
    }
    public int getTemperature(){
        return temperature;
    }
    public static int getActiveCount() {
        return activeCount;
    }
    Run | Debug
    public static void main(String[] args) {
        Thermostat t1=new Thermostat(location: "Living Room",startTemp: 25);
        Thermostat t2=new Thermostat(location: "Bedroom");
        for (int i=0;i<10;i++) {
            t1.raise();
            System.out.println(t1.getTemperature());
        }
        for (int i=0;i<20;i++) {
            t1.lower();
            System.out.println(t1.getTemperature());
        }
        System.out.println(Thermostat.getActiveCount());
    }
}
```

Output:



2. Cinema show (CinemaShow.java, main in this file)

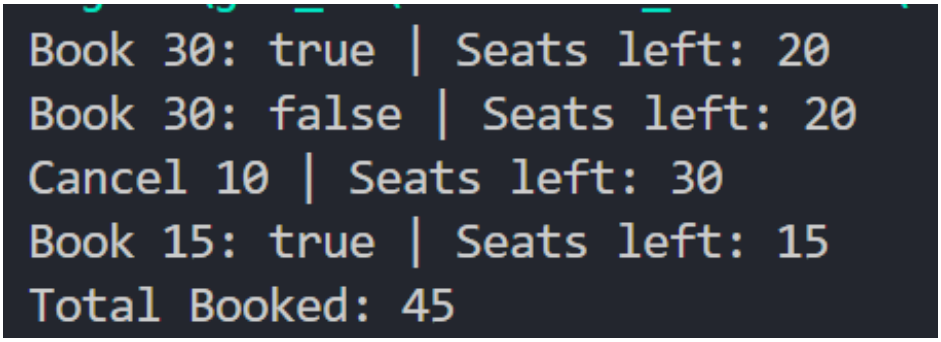
(a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0.
(b) Constructor(title, capacity): set both and seatsAvailable = capacity.
Constructor(title): chain with this(title, 100).
(c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged.
(d) cancel(int n): increase seatsAvailable by n but never above capacity.
(e) Add getSeatsAvailable() and static getTotalBooked().
(f) In main: run a sequence of book and cancel calls, printing the result and seats after

each, then print getTotalBooked().
Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

Solution:

```
public class Cinema_show{
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;
    public Cinema_show(String title,int capacity) {
        this.title=title;
        this.capacity=capacity;
        this.seatsAvailable=capacity;
    }
    public Cinema_show(String title) {
        this(title, capacity: 100);
    }
    public boolean book(int n) {
        if (n<=seatsAvailable) {
            seatsAvailable -=n;
            totalBooked +=n;
            return true;
        }
        return false;
    }
    public void cancel(int n) {
        seatsAvailable+=n;
        if (seatsAvailable>capacity) {
            seatsAvailable=capacity;
        }
    }
    public int getSeatsAvailable() {
        return seatsAvailable;
    }
    public static int getTotalBooked() {
        return totalBooked;
    }
    Run | Debug
    public static void main(String[] args) {
        Cinema_show show = new Cinema_show(title: "Inception", capacity: 50);
        System.out.println("Book 30: " + show.book(n: 30) + " | Seats left: " + show.getSeatsAvailable());
        System.out.println("Book 30: " + show.book(n: 30) + " | Seats left: " + show.getSeatsAvailable());
        show.cancel(n: 10);
        System.out.println("Cancel 10 | Seats left: " + show.getSeatsAvailable());
        System.out.println("Book 15: " + show.book(n: 15) + " | Seats left: " + show.getSeatsAvailable());
        System.out.println("Total Booked: "+Cinema_show.getTotalBooked());
    }
}
```

Output:



3. (For Advanced Learners – home) Parking lot (ParkingLot.java)

(a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0.
(b) park(String type): for “two”, if twoWheelers < twoCap increment and add 20 to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject.
(c) leave(String type): decrement the matching count but never below 0.
(d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue.
Expected output: parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).

Solution:

```java
public class Parking_lot{
    private int two_wheelers=0,four_wheelers=0;
    private final int two_cap,four_cap;
    private static long revenue=0;
    public Parking_lot(int two_cap,int four_cap){
        this.two_cap=two_cap;
        this.four_cap=four_cap;
    }
    public void park(String type){
        if(type.equals(anObject: "two")){
            if(two_wheelers<two_cap){
                two_wheelers++;
                revenue+=20;
                System.out.println(x: "Parked two");
            }else{
                System.out.println(x: "Full");
            }
        }else if(type.equals(anObject: "four")){
            if(four_wheelers<four_cap){
                four_wheelers++;
                revenue+=40;
                System.out.println(x: "Parked four");
            }else{
                System.out.println(x: "Full");
            }
        }
    }
    public void leave(String type){
        if(type.equals(anObject: "two")&&two_wheelers>0){
            two_wheelers--;
            System.out.println(x: "Left two");
        }else if(type.equals(anObject: "four")&&four_wheelers>0){
            four_wheelers--;
            System.out.println(x: "Left four");
        }
    }
    Run | Debug
    public static void main(String[]args){
        Parking_lot lot=new Parking_lot(two_cap: 2,four_cap: 1);
        lot.park(type: "two");
        lot.park(type: "four");
        lot.park(type: "four");
        lot.leave(type: "two");
        lot.park(type: "two");
        System.out.println("two_wheelers:"+lot.two_wheelers+" four_wheelers:"+lot.four_wheelers);
        System.out.println("Revenue:"+revenue);
    }
}
```

Output:

```
Parked two
Parked four
Full
Left two
Parked two
two_wheelers:1 four_wheelers:1
Revenue:80
```

Part B — Project Work (Hour 2): MiniBank

Problem Statement:

In Practical 1 you built the empty menu shell. Now you create the data the bank works on — the Customer and Account classes. Using encapsulation, an account's balance can change only through deposit() and withdraw(), never directly. These two classes are the foundation every later lab extends.

1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as "CUST101". Add a constructor and public getter methods.
2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001").
3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0).
4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance.
5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance.

Solution:

import java.util.Scanner;

```
class Customer {
    private String name;
    private String email;
    private String mobile;
    private final String customerId;

    private static long customerCounter = 101;

    private static String generateCustomerId() {
        return "CUST" + customerCounter++;
    }

    public Customer(String name, String email, String mobile) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.customerId = generateCustomerId();
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }
}

class Account {
    private final String accountNumber;
    private String ownerName;
```

```
    private long balance;
    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber() {
        return String.format("AC%04d", accountCounter++);
    }

    public Account(String ownerName, long balance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = balance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public void deposit(long amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public boolean withdraw(long amount) {
        if (amount > 0 && balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
```

```
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }
}

public class MiniBank {

    record BankInfo(String name, String branch) {}

    enum MenuOption {
        OPEN_ACCOUNT,
        DEPOSIT,
        WITHDRAW,
        TRANSFER,
        EXIT
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        BankInfo bank = new BankInfo("MiniBank", "Main Branch");

        Customer c1 = new Customer("Vaidehi", "vaidehi@gmail.com", "1234567890");

        Account[] accounts = {
                new Account(c1.getName(), 5000),
                new Account("Riya"),
                new Account("Rahul", 10000)
        };

        System.out.println("Customer Details");
        System.out.println("ID: " + c1.getCustomerId());
        System.out.println("Name: " + c1.getName());
        System.out.println("Email: " + c1.getEmail());
```

```
System.out.println("Mobile: " + c1.getMobile());

System.out.println("\nTransactions");

accounts[0].deposit(1000);
accounts[1].deposit(3000);

System.out.println("Withdraw ₹2000 from Account 1: " + accounts[0].withdraw(2000));
System.out.println("Withdraw ₹4000 from Account 3: " + accounts[2].withdraw(4000));

System.out.println("\nAccount Details");
for (Account acc : accounts) {
   System.out.println(
         acc.getAccountNumber() + " | " +
         acc.getOwnerName() + " | Balance = ₹" +
         acc.getBalance()
   );
}

System.out.println("\n================================");
System.out.println(bank.name() + " - " + bank.branch());
System.out.println("================================");

while (true) {
   System.out.println("\nMenu");
   System.out.println("1. Open Account");
   System.out.println("2. Deposit");
   System.out.println("3. Withdraw");
   System.out.println("4. Transfer");
   System.out.println("5. Exit");
   System.out.print("Enter your choice: ");

   int choice = sc.nextInt();

   MenuOption option = switch (choice) {
      case 1 -> MenuOption.OPEN_ACCOUNT;
      case 2 -> MenuOption.DEPOSIT;
      case 3 -> MenuOption.WITHDRAW;
      case 4 -> MenuOption.TRANSFER;
      case 5 -> MenuOption.EXIT;
```

```
            default -> null;
        };

        if (option == null) {
            System.out.println("Invalid choice!");
            continue;
        }

        switch (option) {
            case OPEN_ACCOUNT ->
                System.out.println("Open Account - To be implemented in a later lab.");
            case DEPOSIT ->
                System.out.println("Deposit - To be implemented in a later lab.");
            case WITHDRAW ->
                System.out.println("Withdraw - To be implemented in a later lab.");
            case TRANSFER ->
                System.out.println("Transfer - To be implemented in a later lab.");
            case EXIT -> {
                System.out.println("Thank you for using MiniBank!");
                sc.close();
                return;
            }
        }
    }
  }
}
```

Output:

```
Customer Details
ID: CUST101
Name: Vaidehi
Email: vaidehi@gmail.com
Mobile: 1234567890

Transactions
Withdraw ?2000 from Account 1: true
Withdraw ?4000 from Account 3: true

Account Details
AC0001 | Vaidehi | Balance = ?4000
AC0002 | Riya | Balance = ?3000
AC0003 | Rahul | Balance = ?6000

==============================
MiniBank - Main Branch
==============================

Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 1
Open Account - To be implemented in a later lab.

Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 5
Thank you for using MiniBank!
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```